

Module 4 — Subagents

Lesson 4.1

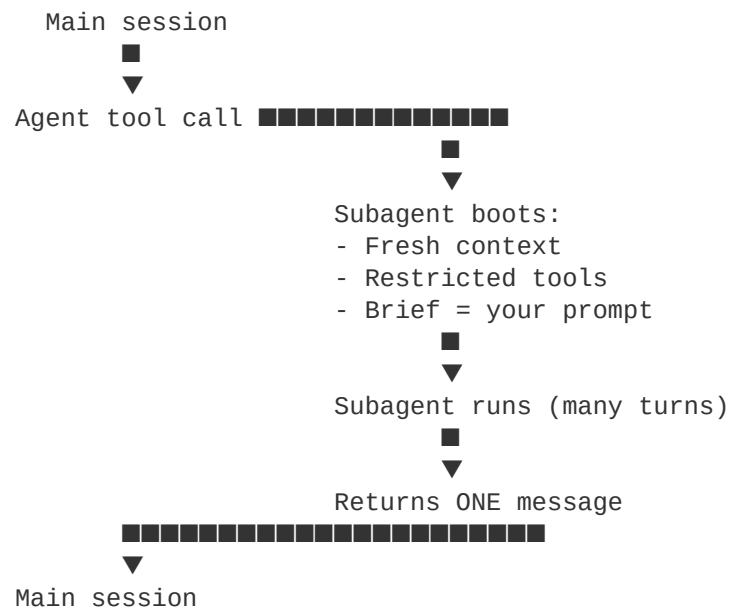
What subagents are and when to use them

A subagent is a child Claude instance with its own context window, its own (often restricted) toolset, and a single self-contained brief. It runs, reports, and returns.

Why it matters

Subagents are the headline power-user feature of Claude Code. They're how you keep long sessions usable, parallelize work, and get independent perspectives. Used badly, they multiply latency and burn budget. Used well, they unlock workflows that aren't possible inline.

The mental model



Three things to internalize:

- 1 **The subagent does not see your conversation.** Its entire context is the brief you wrote. Self-contained or it fails.
- 2 **It returns one message.** No mid-task interaction with you.
- 3 **Its context is invisible to you.** You see only the final report, not the 30 files it read to write it.

When to use a subagent

The decision tree from 2.6, expanded:

Will the task generate a lot of context (search, file reads, log scans)?

YES → delegate to a read-only agent (Explore)

NO → continue

Do you want a perspective independent of your reasoning?

YES → delegate to a fresh agent with no exposure to your conversation

NO → continue

Are there N independent units of work you could run in parallel?

YES → fan out (one agent per unit)

NO → continue

Do you want to attempt a change without polluting your working tree?

YES → delegate with isolation: worktree

NO → continue

→ Just do it inline.

Built-in agent types

Claude Code ships several preset agent types:

Agent	Tools	Use for
<code>general-purpose</code>	All	Catch-all when no specialized agent fits
<code>Explore</code>	Read-only (Read, Glob, Grep, etc.)	Search and reporting without write risk
<code>Plan</code>	All except write/edit	Design implementation plans
<code>claude</code>	All	Same as general-purpose; the default

Beyond these you can define your own (see 4.2).

What's in a subagent's context

When you spawn a subagent, it gets:

- Its system prompt (defined by the agent type)
- Its tool list (also from the agent type)
- The prompt you gave it (and nothing else from your session)
- The working directory and project settings (CLAUDE.md, etc.)

It does **not** get:

- Your conversation history
- Files you've previously read in this session
- Variables you've defined
- Results from earlier subagent runs

Cost considerations

Each subagent invocation is a separate Claude run. It has its own tokens, its own time, its own potential failure. Three subagents in parallel $\approx$ 3x the API spend, but only $\sim$ 1x wall-clock time. For tasks that benefit from concurrency, that's a great trade. For tasks that don't, it's pure waste.

Heuristic: only delegate if you'd otherwise spend significantly more context yourself, OR you genuinely need parallelism, OR independence is the point.

The "brief" matters more than anything

Subagent quality lives or dies by the prompt. We cover this in detail in 4.3 Prompting subagents. The short version: assume your subagent walked into the room cold. Tell it everything it needs.

Try it

- 1 Spawn an Explore agent for "find every place we use the deprecated `localStorage.getItem` API." Compare to doing it inline.
- 2 Spawn two general-purpose agents in parallel — one to draft a CHANGELOG entry, one to draft release notes. Same source data, different outputs.
- 3 Spawn a Plan agent with `isolation: worktree` to attempt a refactor approach. Inspect what it changed without merging.

Gotchas

- **Don't spawn agents for tiny tasks.** Overhead exceeds benefit under ~3 minutes of equivalent work.
- **Don't chain agents with "based on the previous agent's findings."** Each call is fresh; the next agent doesn't see anything. You have to thread the report forward yourself.
- **Read-only agents can't edit.** If you ask Explore to also fix what it finds, you'll get a "cannot use Edit" error.
- **Subagents can be slow.** A research subagent reading 30 files can take a minute. Not a bug — it's doing work.

Further reading

- 4.2 Writing a subagent definition
- 4.3 Prompting subagents effectively
- 4.4 Parallel orchestration
- 4.5 Worktree isolation
- 2.6 When to delegate

Next

→ 4.2 Writing a subagent definition